Software Engineering

Software Processes - Part 1

Okba Tibermacine
National School of Artificial Intelligence

October 15, 2025

Many slides and concepts in this presentation are taken from lecture materials prepared by Prof. Imed Boucherika (SE lectures, 2024, ENSIA)

Outline

- Agile
  - Manifesto
  - Principles
- Agile Software Process Models:
  - RUP
  - Scrum
  - eXtreme Programming
- Choosing a Process Model

From last weeks

Terminologies. . .

- Software Development Process
- Activity
- Task
- Process Model

From last weeks

Software Engineering...

"A discipline that integrates processes, methods and tools for the analysis, design, development, testing and maintenance of computer software products whilst complying with the set of conventional principles and best practices."

From last weeks

- Waterfall Model...
- Spiral
- Incremental
- Prototyping..

From last weeks

Iterative vs. incremental.

Both aim for continuous improvement — but focus differently.

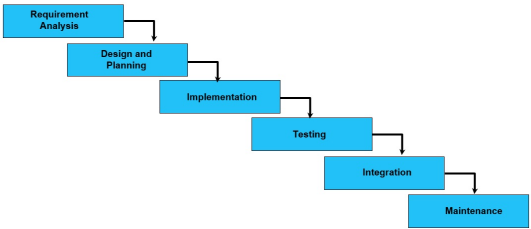
Incremental	**Iterative**
Add new functionality step by step	Refine and improve existing functionality
Build more parts of the system	Make existing parts better
Deliver working increments	Revisit and enhance previous work
• Increment 1: Login • Increment 2: Profile • Increment 3: Cart	• Iteration 1: Basic login • Iteration 2: Faster login • Iteration 3: Better UX

Challenge #1: Requirements

- Often, customers don't know what they want:
  - Feature space is "infinite" or hard to predict
  - The world changes!

Challenge #1: Requirements

- It is no longer feasible to spend:
  - 1 year defining the requirements
  - 1 year designing the system
  - 1 year implementing the system
  - etc
- When the system is ready, it may already be obsolete!



Challenge #2: Detailed Documentation

- Often verbose and of limited use
- In practice, rarely used during implementation
- The plan-and-document approach did not work for software !

Software Process Models: Agile Methodologies

What's Agile ?

- Agile processes are a family of software development methodologies that produce software in **short iterations** and **allow for greater changes in design**.
- Although there is no absolute definition about what constitutes an Agile method, there are several characteristics shared by most Agile methods.
- The philosophy behind agile methods is reflected in the agile manifesto (`http://agilemanifesto.org`) issued by the leading developers of these methods

Agile Methodologies

Software Process Models: Agile Methodologies

Agile Manifesto (2001): Alliance of 17 SE practitioners

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value: core values

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more

Individuals and Interactions over processes and tools

- People and communication matter more than strict procedures or tools.
- Why: Great software comes from collaboration, creativity, and quick feedback, not from blindly following rigid workflows.
- Example:
  - A daily stand-up meeting between developers and clients is more valuable than relying only on a ticketing system.
  - A short team discussion is more valuable than a long report.

Working software over comprehensive documentation

- The Ultimate proof of progress is software that works, not hundreds of pages of documentation.
- Documentation is useful, but can quickly become outdated, while a functioning product provides immediate value and feedback.
- Example:Delivering a prototype users can test is more useful than sending a long specification document.

Customer collaboration over contract negotiation

- Agile teams prefer continuous collaboration with customers instead of sticking rigidly to a signed contract. Continuous collaboration ensures the product meets real needs.
- Requirements change, and direct collaboration ensures the product always meets the real needs of users.
- Example: Adjusting features based on customer feedback during development.

Responding to change over following a plan

- Flexibility is more valuable than rigidly following a plan.
- Why: The world changes — technologies evolve, markets shift, user needs grow. Plans should evolve too.
- Example: Updating priorities mid-project to match new market needs. If the user's priorities change, Agile teams adjust their work instead of insisting on finishing the original plan

Software Process Models: Agile Methodologies

Agile Principles

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Software Process Models: Agile Methodologies

Agile Principles

4. Business people and developers must work together daily throughout the project.

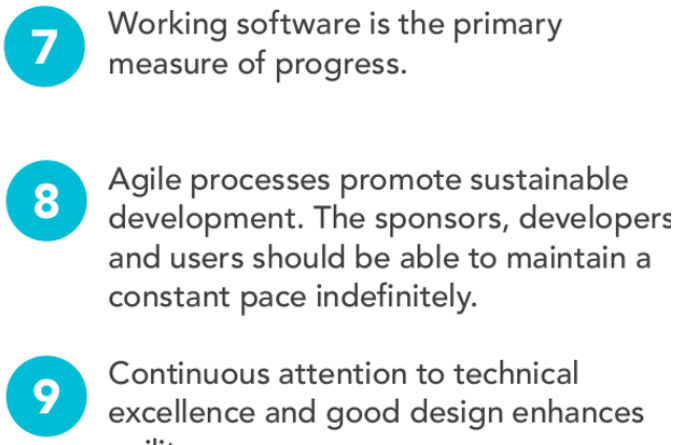
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.

6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.

Software Process Models: Agile Methodologies

Agile Principles

Sustainable pace replaced 40 hours a week notion,... no more long hours, sleepless nights ...



Software Process Models: Agile Methodologies

Agile Principles

Simplicity means focusing on high-value tasks and avoiding or delaying low-value features that may complicate the project

10 Simplicity – the art of maximizing the amount of work not done – is essential.

11 The best architectures, requirements, and designs emerge from self-organizing teams.

12 At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Software Process Models: Agile Methodologies

Agile Principles Rephrased and Explained

01 - Short releases and iterations

- Divide the work into small pieces.
- Release the software to the customer as often as possible.
- Every release/sprint should have a set newer functionalities/bug fixes based on their priorities

Software Process Models: Agile Methodologies

Agile Principles Rephrased and Explained

02 - Incremental design

- Don't try to complete the design up front because not enough is known early about the system.
- Delay design decisions as much as possible, and improve the existing design when more knowledge is acquired.

Software Process Models: Agile Methodologies

Agile Principles Rephrased and Explained

03 - User involvement

- Rather than trying to produce formal, complete, immutable standards at the beginning,
- Ask the users involved with the project to provide constant feedback.
- Customers should be closely involved throughout the development process.

Software Process Models: Agile Methodologies

Agile Principles Rephrased and Explained

04 - Embrace change

Expect the system requirements to change, and so design the system to accommodate these changes.

Software Process Models: Agile Methodologies

Agile Principles Rephrased and Explained

05 - People, not process

- The skills of the development team should be recognized and exploited.
- "Team members should be left to develop their own ways of working without prescriptive processes."
- But remember,..

Software Process Models: Agile Methodologies

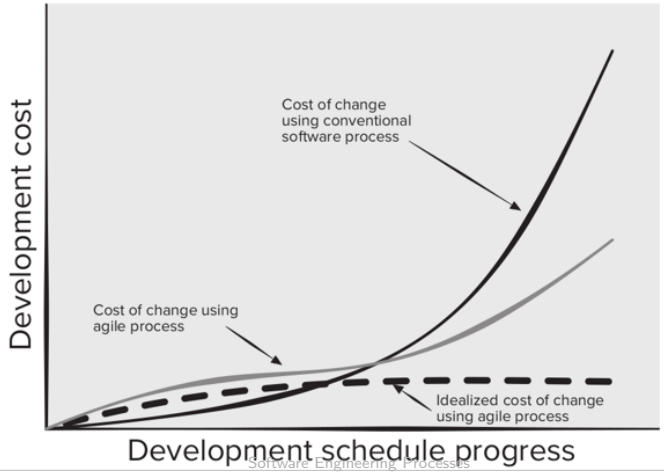
Agile Principles Rephrased and Explained

06 - Maintain simplicity

- Focus on simplicity in both the software being developed and in the development process including the communication activity
- Actively work to eliminate complexity from the system.

Software Process Models: Agile Methodologies

Agile Processes vs Traditional Processes when it comes to Handling Changes



Rational Unified Process

Software Process Models: Rational Unified Process

- The Rational Unified Process (RUP) is a software process framework developed by Rational Software Corporation, which was acquired by IBM
- This process framework is driven by three major concepts
  - Use-case and requirements driven
  - Architecture centric
  - Iterative and incremental

RUP is iterative and incremental in that it promotes large software to be developed in smaller pieces or increments.

Software Process Models: Rational Unified Process

Phases of Rational Unified Process:

- Not named after the activities such as design, testing, or coding;
- An iteration may include many activities in varying degrees.
- There are four phases in RUP:
  - Inception
  - Elaboration
  - Construction
  - Transition

Software Process Models: Rational Unified Process

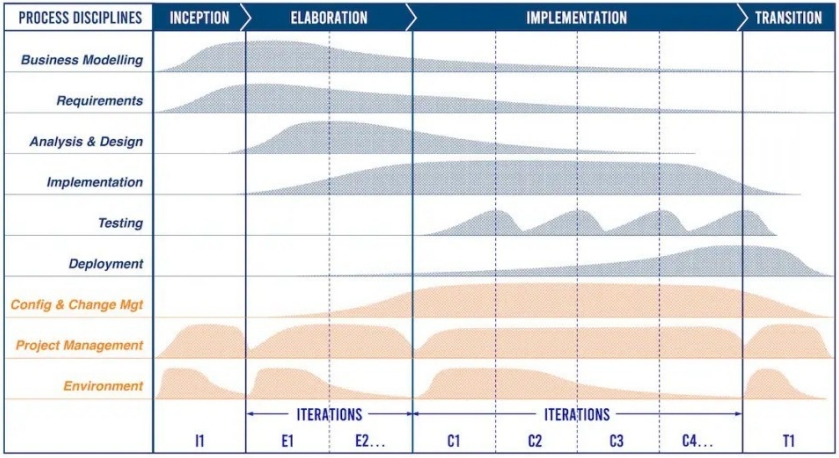
Phases of

- Not na
- An iter
- There
  - In
  - El
  - Co
  - Transition

- As an increment of the product is developed, it may go through several iterations within each phase.
- The degree of the activities such as requirements specifications, testing, or coding that takes place within each phase is also different.

Software Process Models: Rational Unified Process

Phases of Rational Unified Process:



Software Process Models: Rational Unified Process

Inception Phase: is a planning phase that includes the following primary tasks to establish:

- The scope and clarify the goals of the software project.
- The critical use cases and the major scenarios that will drive the architecture and design (including design alternatives)
- The schedule and required resources
- Planning for the implementation, testing, integration, and configuration methodologies.
- Estimate the potential risks.

Software Process Models: Rational Unified Process

Elaboration Phase: most critical phase of the RUP as most of the unknowns should be resolved. The primary objectives are to establish:

- All the major and critical requirements for the system.
- The baseline design.
- Platforms and methodologies for the implementation, test, and integration.
- Major test scenarios.
- Measurement and metrics for the agreed goals.

Software Process Models: Rational Unified Process

Construction Phase: is equal to the production phase in manufacturing. The following objectives are the key points of this phase:

- Complete most of the implementation in a timely manner..
- Achieve the version of the code that is releasable to a restricted set of users.
- Establish the remaining activities that need to be completed to achieve the goals of the project.

Software Process Models: Rational Unified Process

Transition Phase:

- is the last phase prior to the release of the software to general users. All the fixes and components are integrated.
- The noncode artifacts, such as manuals and educational materials, are also integrated into the complete product.

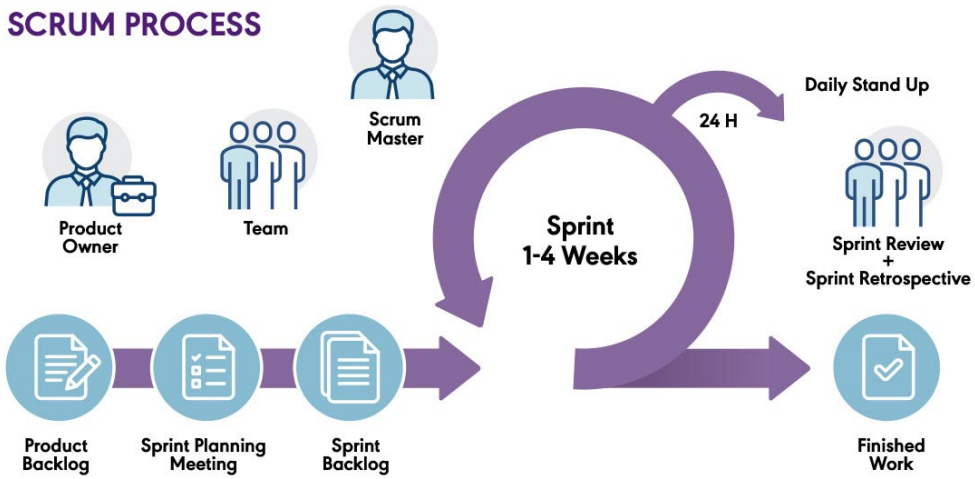
Software Process Models: Rational Unified Process

Transition Phase: The objectives of this phase are the following:

- Establish the final software product for general release.
- Establish user readiness and acceptance of the software.
- Establish support readiness.
- Establish strategy for release, deployment, migration and possible change of business procedures..

Scrum

Scrum Principles for Software Projects



Scrum Principles for Software Projects

Scrum: Definition

- Is a project management framework or even a methodology composed of a set of principles for creating, improving and delivering high-quality products to customers in an iterative and incremental way.
- It emphasizes on teamwork, collaboration, communication, transparency and continual improvement among team members $\Rightarrow$ (**Self-organized Teams**)
- Scrum is built upon by the collective intelligence of the people using it. Rather than provide people with detailed instructions, the rules of Scrum guide the relationships and interactions

Scrum Principles for Software Projects

Scrum: Why?

It is designed to deliver value to the customer and satisfy their needs throughout the development of the project by:

- Selecting and prioritizing the right functionalities at the right increment of delivery
- Creating an environment featured by transparency in communication, collective responsibility and continuous progress

Scrum Principles for Software Projects

Scrum: Why?

- The product is broken down into a set of manageable and understandable chunks/deliverables
- Unstable requirements do not hold up progress. (Complex features, can wait, whilst we can deliver incrementally new features)
- Customers see on-time delivery of increments and gain feedback on how the product works
- The whole team has visibility of everything, and consequently team communication and morale are improved

Scrum Principles for Software Projects

Scrum: Why?

- The product is broken down into a set of manageable and understandable chunks
- Unstab vait, whilst we can
- Custom the product works
- The whole team has visibility of everything, and consequently team communication and morale are improved

More suited to the current trends of applications in this era. Startup Apps, Mobile Apps... are mostly needed in a short time of few months (not years)

Scrum Principles for Software Projects

Scrum: Term

- Waterfall Model Analogy to the Relay race
  - Each athlete (racer) will do his phase/lap.
  - In the next phase, they will pass the baton (stick) to the next racer
- Scrum as a word is drawn from the Rugby game indicating Restarting a play/game after stoppage due to a foul or ...
  - Analogy for Software Development, The developers work as a team during each iteration, afterwards, all members restart fresh from the same point.

Scrum Principles for Software Projects

Scrum members:

- **Committed (Scrum Team)**
  - are members who are fully committed to the success of the project.
  - Smaller teams of 10 members or less to avoid gaps in communication and decreased productivity.
  - Is Accountable for creating a valuable increment at every sprint.
- **Involved**
  - This includes members who have interest in the success of the project including executives, customers...

Scrum Principles for Software Projects

Scrum Roles for Committed members (Scrum Team):

- Product Owner
- Scrum Master
- Development Team

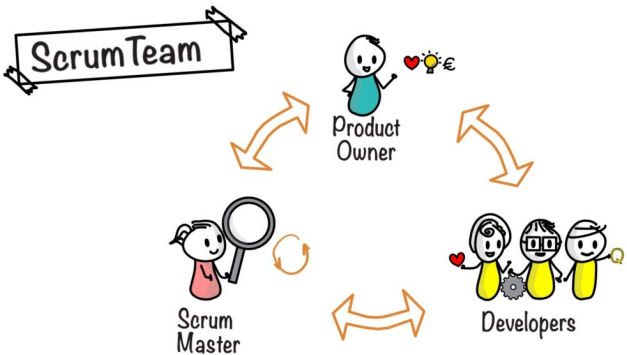


Image Source: Google

Scrum Principles for Software Projects

Scrum Roles: Product Owner [Must be Single Person, not committee of people]

- Responsible for setting the vision of the product and its primary goal.
- Communicate with the clients, end-users and stakeholders to understand their needs and get feedback from them.
- Have an insight on how to increase the value of the product
- Is accountable for managing the product backlog
  - Creating, Ordering and Communicating product backlog items
  - Ensuring that the product backlog is transparent, visible and understood.
- Approves the delivered product sprint by validating it is meeting the intended value.

Scrum Principles for Software Projects

Scrum Roles:

Scrum Master (Servant Leadership)

He/She is the facilitator of the team ensuring that:

- Smooth communication between the product owner the development team
- Team is progressing on time and meeting deadlines to deliver an increment of the product and remove any obstacle hindering the progress.
- Team is following the scrum methodology throughout the entire process:
  - Train, teach, lead daily meetings and provide feedback

Scrum Principles for Software Projects

Scrum Roles:

Development Team

- They execute the sprint under the vision of the product owner and the direction of the scrum master.
- Consists of cross-functional members (with specialized skills) with the responsibility to turn the Product Backlog items they select into an Increment of useful and valuable product functionality
- They are responsible for managing the progress of work during a Sprint

Cross-functional: members from different areas/departments

Scrum Principles for Software Projects

Scrum Artifacts:

- Product Backlog
- Spring Backlog
- Product Increment

Scrum Principles for Software Projects

Scrum Artifacts: Product Backlog

- List of all features, functions, requirements, enhancements, and fixes to be made to the product in future releases.
- Product Backlog items:
  - Have the attributes of a description, order, estimate duration and value.
  - Include test descriptions that will prove its completeness when “Done”.
- Product Backlog refinement is the act of breaking down and further defining Product Backlog items into smaller more precise items
- Maintained and managed by the Product Owner.

Scrum Principles for Software Projects

Scrum Artifacts: Product Backlog

Items are usually inserted in the form of:

- **User Story:** is an informal statement to express a small feature from the perspective of an end-user whilst clearly showing the value.
  - As a (Website Visitor), I want to (login), so that I can (access my expenses)
  - As a (Website Visitor), I want to (reset password), so that I can (regain access)

- **Action Task:** Corresponding to an action item to be executed.
  - Implement UI for login with minimal design.
  - Implement BL to verify authentication against internal DB

- **Epic:** Set of stories that would correspond to a major feature/component to be implemented.

Scrum Principles for Software Projects

Scrum Artifacts:

Sprint Backlog:

- Is composed of Spring Goal + a list of all the work items (usually TASKS) that must be done by the end of the sprint timebox
- Timebox: is the time frame/duration that should not be exceeded to complete a task/sprint/meeting ...
  - A sprint timebox is ranging from 2 weeks to 4 weeks..
- Tasks are taken from the product backlog, prioritized and assigned to members of the development team during the sprint meeting.
- Depending on the state of the project, tasks may be added to, removed from, or re-prioritized in the sprint backlog

Scrum Principles for Software Projects

Scrum Artifacts: Product Increment

is the set of all the tasks, use cases, user stories, product backlogs and any element that:

- is developed during the sprint(s)
- is marked as completed based on the “done definition”
- will be made available to the end user in the form of Software.

Product increments should build upon each other until the team completes the project.

Scrum Principles for Software Projects

Scrum Ceremonies (Events):

- Are meetings or events to inspect and adapt Scrum artifacts with the aim to improve transparency.
- To reduce complexity, Scrum events are held at the same time and same place.
- List of Scrum Events
  - Sprint Planning
  - Daily Scrum
  - Sprint Review
  - Sprint Retrospective

Scrum Principles for Software Projects

Scrum Ceremonies: Sprint Planning

- Take place at the start of the Sprint, Its aim is to select items from the Product Backlog to be implemented in the current Sprint.
- Attended by all members and chaired by the PO to discuss the most important items.
- Three Points in this event:
  - Why is this Sprint Valuable?
  - What can be done in this Sprint?
  - How will the chosen work get done?

Scrum Principles for Software Projects

Scrum Ceremonies: Daily Scrum

- Daily brief meetings of 15 minutes timebox for developers (no need for PO and SM to attend, unless they are developers too).
- The aim is for every member to stand up and speak about:
  - What did I do yesterday?
  - What will I do today?
  - Are there any impediments in my way?

Scrum Principles for Software Projects

Scrum Ceremonies: Sprint Review

- **is to inspect the outcome of the Sprint and determine future adaptations.**
- The Scrum Team presents the results of their work to key stakeholders and progress toward the Product Goal is discussed.
- Attendees collaborate on what to do next. The Product Backlog may also be adjusted to meet new opportunities.
- The event is timeboxed to a maximum of four hours for a one-month Sprint

Scrum Principles for Software Projects

Scrum Ceremonies: Sprint Retrospective

- The aim of the Sprint Retrospective is to plan ways to increase quality and effectiveness.
- The Scrum Team inspects how the last Sprint went with regards to individuals, interactions, processes, tools, and their Definition of Done.
- It is timeboxed to a maximum of three hours for a one-month Sprint.

Scrum Principles for Software Projects

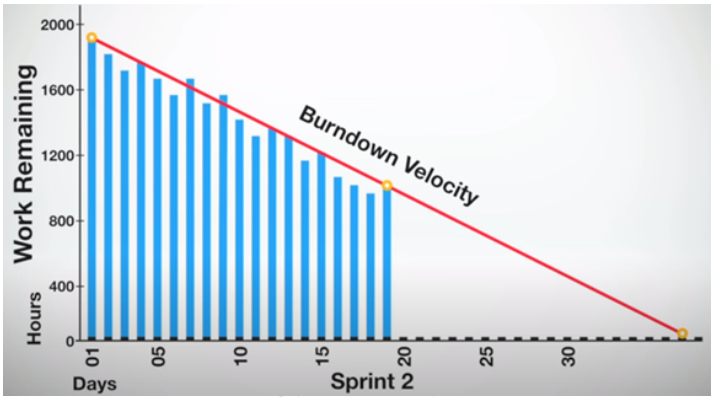
Scrum Burndown Chart:

- Items are assigned an estimated duration in terms of hours. (Or even points reflecting the complexity of the task.)
- At each sprint:
  - At day n
  - We compute the number of estimated hours left.
  - Whenever developers work on an item, they update the progress (estimated hours left)

Scrum Principles for Software Projects

Scrum Burndown Chart: Can be used to predict if a sprint can be completed on time by computing the Burndown Velocity (work Completed

time elapsed)



Scrum Principles for Software Projects

Five Values of Scrum:

- **Commitment:** Members commits to achieving its goals and to supporting each other.
- **Focus:** Their primary focus is on the work of the Sprint to make the best possible progress toward these goals
- **Openness:** The Scrum Team and its stakeholders are open about the work and the challenges (they share progress, information. . .)
- **Respect:** Scrum Team members respect each other to be capable, independent people, and are respected as such by the people with whom they work
- **Courage:** The Scrum Team members have the courage to do the right thing, to work on tough and challenging problems.

Scrum Principles for Software Projects

Steps of How to get started using Scrum:

- Setup the tools/platform for using Scrum (Jira/Google Calendar/Slack/..)
- As a team, Assign
  - A member as the Product owner for the project (Someone who is the visionary of the project)
  - A member as Scrum Master (Someone with better skills in leading, management and knowledgeable).
  - Rest of the team, are the development team (including even the PO + SM)
- Product owner will be gathering the requirements and turning them into tasks/stories inside the product backlog
- The scrum master needs to define and fix the timing for the ceremonies
  - Sprint Planning: 1st Monday or Tuesday or.. of every month. / or every two weeks

Scrum Principles for Software Projects

Steps of How to get started using Scrum:

Iteratively:

- Hold the Sprint Planning Meeting:
  - Discuss the items to execute in the sprint,
  - Assign them to developes
- Sprint Execution:
  - Daily Scrum Meeting
  - Development
  - Integration
  - Testing
  - Acceptance based on Done Definition
- Sprint Review & then Sprint Retrospective Meeting.

Scrum Simulation: Building MealPrep Pro

Phase 1: The Genesis

Step 1: Define the Product Vision

MealPrep Pro Vision

For busy professionals who struggle with healthy eating, MealPrep Pro is a mobile app that simplifies meal planning and grocery shopping. Unlike generic recipe apps, our product intelligently generates personalized weekly meal plans and creates optimized grocery lists, making healthy eating effortless.

Phase 1: The Genesis

Step 2: Meet the Scrum Team

- **Omar (Product Owner):** Manages the Product Backlog and represents stakeholders.
- **Samir (Scrum Master):** A servant-leader who facilitates events and removes impediments.
- **The Developers:** A cross-functional team.
  - **Dalia:** Frontend specialist (UI/UX)
  - **Daoud:** Backend developer (APIs/Databases)
  - **Douaa:** Mobile development expert (iOS/Android)

Phase 1: The Initial Product Backlog

Omar creates the initial Product Backlog, prioritizing user stories by value:

- **PBI-001 (HIGH):** As a user, I want to create an account and set dietary preferences. *(5 Story Points)*
- **PBI-002 (HIGH):** As a user, I want to browse a library of healthy recipes. *(8 Story Points)*
- **PBI-003 (HIGH):** As a user, I want to add recipes to my weekly meal plan. *(5 Story Points)*
- **PBI-004 (HIGH):** As a user, I want the app to automatically generate a grocery list. *(8 Story Points)*
- **PBI-005 (MEDIUM):** As a user, I want to mark pantry items so I don't buy duplicates. *(3 Story Points)*
- **PBI-006 (MEDIUM):** As a user, I want to see which local stores have my items in stock. *(13 Story Points)*
- **PBI-007 (LOW):** As a user, I want to adjust serving sizes for recipes. *(5 Story Points)*

Phase 2: Sprint 1 (2-Week Sprint)

Scrum Ceremonies: Sprint Planning

Part 1: The "What" - Selecting the Work

- Omar presents the top priority items from the Product Backlog.
- The team discusses the requirements for each.
  - **Daoud on PBI-001:** "Should dietary preferences allow multiple selections?"
  - **Omar:** "Yes, absolutely. Multiple selections with checkboxes."
  - **Douaa on PBI-002:** "How many recipes do we need in the initial library?"
  - **Omar:** "Let's start with 30 diverse recipes."
- After discussion, the team feels confident they can complete PBI-001, PBI-002, and PBI-003.

Phase 2: Sprint 1 (2-Week Sprint)

Scrum Ceremonies: Sprint Planning

Part 1: The "What" - Selecting the Work

Sprint 1 Goal

By the end of this Sprint, a user can successfully register with dietary preferences, browse a library of recipes with nutritional information, and create a personalized weekly meal plan.

Phase 2: Sprint 1 (2-Week Sprint)

Scrum Ceremonies: Sprint Planning

Part 2: The "How" - Creating the Sprint Backlog

The Developers decompose the selected User Stories into specific tasks:

PBI-001: User Registration

- Design registration UI (Dalia)
- Implement email auth (Daoud)
- Set up user DB schema (Daoud)
- Integrate frontend/backend (Douaa)

PBI-002: Recipe Library

- Design recipe card layout (Dalia)
- Create recipe DB schema (Daoud)
- Build recipe API (Daoud)
- Add filtering by preference (Douaa)

Phase 2: Sprint 1 (2-Week Sprint)

Scrum Ceremonies: Sprint Planning

Part 2: The "How" - Creating the Sprint Backlog

The Developers decompose the selected User Stories into specific tasks:

PBI-003: Weekly Meal Plan

- Design calendar view UI (Dalia)
- Build meal plan CRUD API (Daoud)
- Implement persistence (Douaa)
- Add remove-meal feature (Douaa)

The team estimates each task will take 4-16 hours. They create a Sprint Backlog board (digital or physical) with columns: To Do, In Progress, and Done.

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 3)

Every morning, the Developers meet for 15 minutes to inspect progress toward the Sprint Goal.

- **Dalia:** "Yesterday, I completed the registration form UI. Today, I'll finish the recipe card layout. No impediments."

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 3)

Every morning, the Developers meet for 15 minutes to inspect progress toward the Sprint Goal.

- **Dalia:** "Yesterday, I completed the registration form UI. Today, I'll finish the recipe card layout. No impediments."
- **Daoud:** "Yesterday, I set up the user database. Today, I'll work on password hashing and start the recipe database schema. **Impediment:** I'm concerned about the nutritional data format we need."

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 3)

Every morning, the Developers meet for 15 minutes to inspect progress toward the Sprint Goal.

- **Dalia:** "Yesterday, I completed the registration form UI. Today, I'll finish the recipe card layout. No impediments."
- **Daoud:** "Yesterday, I set up the user database. Today, I'll work on password hashing and start the recipe database schema. **Impediment:** I'm concerned about the nutritional data format we need."
- **Samir (Scrum Master):** *(Takes note)* "Got it. I'll talk to Omar right after this and get back to you within the hour."

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 3)

Every morning, the Developers meet for 15 minutes to inspect progress toward the Sprint Goal.

- **Samir (Scrum Master):** *(Takes note)* "Got it. I'll talk to Omar right after this and get back to you within the hour."
- **Douaa:** "Yesterday, I wrote unit tests for authentication. Today, I'll start integrating the registration frontend with the backend API. No blockers."

After the Daily Scrum, Samir immediately connects with Omar to clarify the nutritional data format. Within 30 minutes, he shares the standardized format (calories, protein, carbs, fat, fiber) with Daoud, removing the impediment.

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 7 - Week 2)

The team synchronizes, identifies impediments, and adapts their plan for the day.

- **Dalia:** "Yesterday, I completed the weekly calendar view and it looks great! Today, I'm implementing the drag-and-drop functionality for adding recipes to specific days. No impediments."

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 7 - Week 2)

The team synchronizes, identifies impediments, and adapts their plan for the day.

- **Dalia:** "Yesterday, I completed the weekly calendar view and it looks great! Today, I'm implementing the drag-and-drop functionality for adding recipes to specific days. No impediments."
- **Daoud:** "Yesterday, I finished all the API endpoints for recipe browsing and meal planning. Today, I'll seed the database... **Impediment:** three of the recipes Omar sent are missing serving size information."
  - *Samir: "I'll check with Omar immediately and get you that information."*

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 7 - Week 2)

- **Douaa:** "Yesterday, I completed the integration between registration and authentication. Today, I'm working on the recipe filtering... **Blocker:** There's a bug where vegetarian recipes still show meat ingredients - I'm blocked until Daoud finishes seeding the database so I can test with real data."

Phase 2: Sprint 1 in Progress

Scrum Ceremonies: Daily Scrum (Day 7 - Week 2)

- **Douaa:** "Yesterday, I completed the integration between registration and authentication. Today, I'm working on the recipe filtering... **Blocker:** There's a bug where vegetarian recipes still show meat ingredients - I'm blocked until Daoud finishes seeding the database so I can test with real data."
- **Daoud:** *(Responding to Douaa)* "I'll prioritize the database seeding this morning so you can test by this afternoon."
  - *Douaa: "Perfect, thanks!"*

Outcome: Inspect & Adapt in Action

The team adjusts their plan for the day based on the discussion. Daoud re-prioritizes his work to unblock Douaa, demonstrating the self-organizing and collaborative nature of the Developers.

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

The Scrum Team presents the working Increment to key stakeholders.

The Demonstration:

- **Dalia shows the UI:** "I'll create a new account... now I select my dietary preferences - vegetarian and gluten-free."

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

The Scrum Team presents the working Increment to key stakeholders.

The Demonstration:

- **Dalia shows the UI:** "I'll create a new account... now I select my dietary preferences - vegetarian and gluten-free."
- **Douaa shows functionality:** "Because we selected 'vegetarian,' the app only shows meat-free recipes. Let's look at the 'Grilled Vegetable Pasta'..."

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

The Scrum Team presents the working Increment to key stakeholders.

The Demonstration:

- **Dalia shows the UI:** "I'll create a new account... now I select my dietary preferences - vegetarian and gluten-free."
- **Douaa shows functionality:** "Because we selected 'vegetarian,' the app only shows meat-free recipes. Let's look at the 'Grilled Vegetable Pasta'..."
- **Daoud shows the result:** "I'll add this recipe to Monday lunch on our weekly meal plan. And here is our personalized plan for the week!"

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

Stakeholder Feedback and Product Backlog Adaptation:

- **Fitness Coach:** "This is fantastic! One question - can users see the total calories for the entire week?"
  - *Omar adds PBI-008 to the Product Backlog.*

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

Stakeholder Feedback and Product Backlog Adaptation:

- **Fitness Coach:** "This is fantastic! One question - can users see the total calories for the entire week?"
  - *Omar adds PBI-008 to the Product Backlog.*
- **Nutritionist:** "I love the filtering. Can we add a 'dairy-free' option?"
  - *Omar updates PBI-001 with this new requirement.*

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Review

Stakeholder Feedback and Product Backlog Adaptation:

- **Fitness Coach:** "This is fantastic! One question - can users see the total calories for the entire week?"
  - *Omar adds PBI-008 to the Product Backlog.*
- **Nutritionist:** "I love the filtering. Can we add a 'dairy-free' option?"
  - *Omar updates PBI-001 with this new requirement.*
- **Potential User:** "I can't tell which meal is breakfast, lunch, or dinner."
  - *Omar creates PBI-009: "As a user, I want to categorize meals..."*

The Sprint Review concludes with everyone aligned on what was delivered and what to do next.

Sprint Retrospective: Inspecting the Process

Date: Friday late afternoon, immediately after Sprint Review

Duration: 1.5 hours

Attendees: Omar, Samir, Dalia, Daoud, and Douaa (private team meeting)

This is the team's opportunity to inspect their process and plan improvements. Samir facilitates using the "Start, Stop, Continue" format.

WHAT SHOULD WE START DOING?

- **Douaa:** "I think we should start doing more pair programming, especially when integrating frontend and backend. Dalia and I spent 3 hours debugging an API integration issue that could have been solved in 30 minutes if we worked together."
  - *Team: Nods in agreement.*

Sprint Retrospective: Inspecting the Process

WHAT SHOULD WE START DOING?

- **Daoud:** "We should start refining the Product Backlog items one Sprint ahead. Some of the acceptance criteria weren't crystal clear until we were already building."
  - **Omar:** "I agree. I'll make sure the top items are well-defined before each Sprint Planning."

Sprint Retrospective: Discussion

WHAT SHOULD WE STOP DOING?

- **Dalia:** "We should stop underestimating the UI polish time. The recipe cards took twice as long as expected because we kept tweaking the design. Maybe we need a design review before we start coding?"
  - **Samir:** "Good insight. Let's explore that."

Sprint Retrospective: Discussion

WHAT SHOULD WE STOP DOING?

- **Dalia:** "We should stop underestimating the UI polish time. The recipe cards took twice as long as expected because we kept tweaking the design. Maybe we need a design review before we start coding?"
  - **Samir:** "Good insight. Let's explore that."

Sprint Retrospective: Discussion

WHAT SHOULD WE CONTINUE DOING?

- **Daoud:** "The Daily Scrums have been really valuable. Let's keep them focused and time-boxed like we've been doing."

Sprint Retrospective: Discussion

WHAT SHOULD WE CONTINUE DOING?

- **Daoud:** "The Daily Scrums have been really valuable. Let's keep them focused and time-boxed like we've been doing."
- **Douaa:** "I want to continue the energy we had this Sprint. Everyone was collaborative and willing to help when someone was blocked."

Sprint Retrospective: Discussion

WHAT SHOULD WE CONTINUE DOING?

- **Daoud:** "The Daily Scrums have been really valuable. Let's keep them focused and time-boxed like we've been doing."
- **Douaa:** "I want to continue the energy we had this Sprint. Everyone was collaborative and willing to help when someone was blocked."
- **Omar:** "The Sprint Goal really helped us stay focused. When we were tempted to add features, we asked, 'Does this help us achieve the Sprint Goal?' That kept us on track."

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Retrospective **Actionable Commitments for Sprint 2:**

The team agrees on specific improvements for the next Sprint.

1. **Pair Programming:** Schedule at least two 2-hour pair programming sessions for complex integrations.
2. **Backlog Refinement:** Hold a 1-hour refinement session mid-Sprint to prepare for the next Sprint Planning.
3. **Design Reviews:** Dalia will create design mockups for UI-heavy stories, and the team will review them together before Sprint Planning.
4. **Task Estimation:** Break tasks down further so nothing is estimated to be larger than 8 hours.

Phase 2: Sprint 1 Conclusion

Scrum Ceremonies: Sprint Retrospective **Actionable Commitments for Sprint 2:**

The team agrees on specific improvements for the next Sprint.

1. **Pair Programming:** Schedule at least two 2-hour pair programming sessions for comple
2. **Backl** repare for the next Sprint Planning.
3. **Design Reviews:** Dalia will create design mockups for UI-heavy stories, and the team will review them together before Sprint Planning.
4. **Task Estimation:** Break tasks down further so nothing is estimated to be larger than 8 hours.

Samir documents these commitments to ensure they are followed next sprints.

Phase 3: The Cycle Continues - Sprint 2

Scrum Ceremonies: Sprint Planning

The team gathers to plan the next Sprint, incorporating feedback and their new process improvements.

Top Priorities (from adapted Product Backlog):

- **PBI-004:** Automatic grocery list generation
- **PBI-009:** Categorize meals as breakfast, lunch, dinner
- **PBI-008:** Weekly nutritional totals view

Sprint 2 Goal

By the end of this Sprint, users can generate an intelligent grocery list from their meal plan, categorize meals by type, and view their weekly nutritional totals.

The Sprint begins!

Simulation Summary: Roles in Action

- **Omar (Product Owner)**
  - Defined the vision, created and prioritized the Product Backlog, engaged stakeholders, and made tough trade-off decisions on what to build next.
- **Samir (Scrum Master)**
  - Facilitated all events, removed impediments (data format), coached the team on Scrum, and fostered continuous improvement through retrospectives.
- **The Developers (Dalia, Daoud, Douaa)**
  - Self-organized to accomplish the Sprint Goal, created the product Increment, ensured quality, and committed to process improvements.

Simulation Conclusion: The Power of Scrum

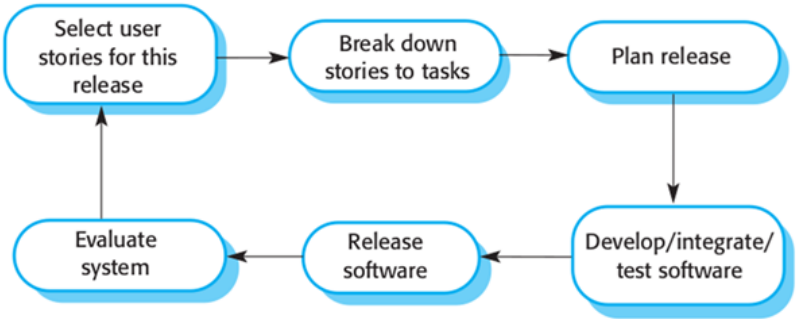
Through the journey of building MealPrep Pro, the team demonstrated how Scrum enables:

- **Focus** $\Rightarrow$ Clear Sprint Goals kept the team aligned.
- **Adaptation** $\Rightarrow$ Feedback loops from the Sprint Review allowed for quick course corrections.
- **Transparency** $\Rightarrow$ Everyone knew what was being built and why.
- **Collaboration** $\Rightarrow$ Daily communication and cross-functional teamwork delivered complete features.
- **Continuous Value** $\Rightarrow$ Working, valuable software was delivered every 2 weeks.
- **Improvement** $\Rightarrow$ Regular retrospectives made the team better and more efficient each Sprint.

eXtreme Programming (XP)

eXtreme Programming (XP)

XP Process Model



eXtreme Programming (XP)

XP Process Model

- XP is an acronym for eXtreme Programming
- The most widely used agile process
- This approach uses user stories or scenarios for requirements, test-driven development, has direct customer involvement on the team (typically defining acceptance tests), uses pair programming, and provides for continuous code refactoring and integration.

eXtreme Programming (XP)

XP Process Model

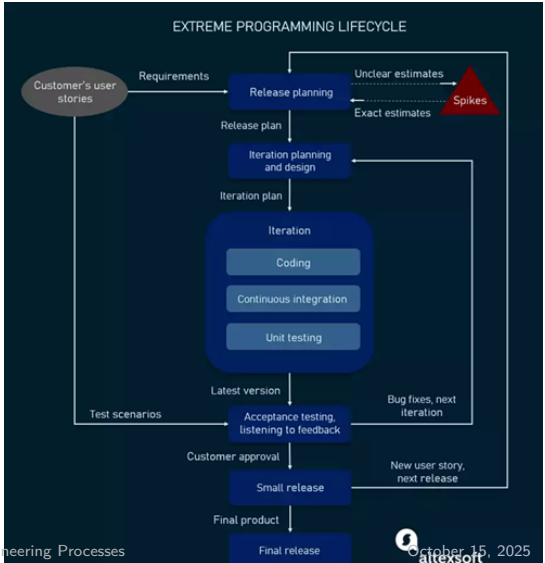
Why Pair Programming:

- It supports the idea of collective ownership and responsibility for the system
- It acts as an informal review process because each line of code is looked at by at least two people
- It encourages refactoring to improve the software structure, readability of source code and even maintainability

eXtreme Programming (XP)

XP Life Cycle

Image copied from
https://content.altexsoft.com/



eXtreme Programming (XP)

eXtreme Programming (XP) Principles:

1. Code Standard
2. Refactoring
3. Pair Programming
4. Test-Driven Development
5. Collective Code Ownership
6. Continuous Integration
7. Small Release
8. Onsite Customer
9. Sustainable Pace
10. System Metaphor
11. Simple Design
12. Planning Game

Principle 1: Code Standard

Coding Standards

Conventional recommendations of the coding style to ensure consistency, readability of code and maintenance.

This includes:

- Naming of variables
- Use of exceptions
- SQL syntax
- Indentation
- Constructing classes

Principle 2: Refactoring

Definition

Refactoring is the process to change and restructure the code without changing its behaviour (functionalities).

The aim of refactoring is:

- Improve readability and maintenance
- No redundancy (removing duplicate business logic)
- Ensure all variables in scope, defined and used
- No long functions or methods
- Removing unnecessary business logic (BL)
- Proper use of access modifiers etc.

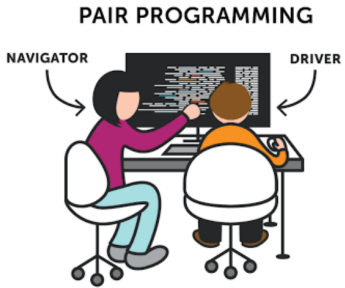
Principle 3: Pair Programming

Definition

Pair Programming consists of two programmers operating on the same code and unit test cases, on the same system.

How it works:

- The first developer focuses on coding
- The other one reviews and inspects code, suggests improvements, and fixes mistakes along the way.



Principle 4: Test-Driven Development (TDD)

Definition

Automated unit tests are written and created **before** the code itself.

Key Points:

- Every piece of code must pass the test created initially
- TDD allows programmers to use immediate feedback to produce reliable software

Principle 5: Collective Code Ownership

Definition

The whole team is responsible for the design and building of the software.

Characteristics:

- Each member who has access to the code can review and update code
- Collective code ownership encourages the team to cooperate more and feel free to bring new ideas and new features

Principle 6: Continuous Integration (CI)

Definition

Developers do pair programming on **local versions** of the code → There is a need to integrate changes made every few hours or on a daily basis.

Process:

- After every code compilation and build, integrate it
- All tests are executed automatically for the entire project
- This is also known as **Continuous Delivery**

Principle 7: Small Release

Definition

XP teams release the MVP quickly and further keep developing the product by making small and incremental updates/releases.

Benefits:

- Allows developers to frequently receive feedback
- Detect bugs early
- Monitor how the product works in production

Principle 8: Onsite Customer

Definition

This is a similar role as Product Owner in Scrum.

Responsibilities:

- Customers (or even End-users) should fully participate in the development
- The customer should be present all the time to:
  - Answer team questions
  - Craft the vision
  - Set priorities
  - Plan release
  - Define user stories and acceptance criteria
  - Resolve disputes if necessary

Principle 9: Sustainable Pace

Definition

No overtime for employees.

Benefits:

- The process of refactoring and reviewing code can help staff to relax
- Be at the same pace
- Refresh their minds about what's done

Principle 10: System Metaphor

Definition

A story that everyone - customers, programmers, and managers - can tell about how the system works.

Implementation:

- Naming concepts for classes and methods are used
- Make it easy for a team member to guess the functionality of a particular class/method from its name only

Principle 11: Simple Design

Definition

The team will not do complex or big architecture and designs upfront; instead the team will start with a simple design and let it emerge and evolve over a period of iterations.

Approach:

- The team ensures to release a working MVP initially
- Keep adding new features in the coming iterations

Principle 12: Planning Game

Definition

There will be a meeting that occurs at the beginning of an iteration cycle.

Process:

- The development team and the customer get together
- Discuss and approve a product's features
- At the end of the planning game:
  - Developers plan for the upcoming iteration and release
  - Assigning tasks for each of them

Benefits and Drawbacks

Benefits of XP

- Stable System
- Clear and clean code
- Fast delivery of features
- No overtime
- Better Collaboration
- Good transparency and visibility
- Customer Satisfaction

Drawbacks of XP

- Unclear Estimates (hours, budget, duration...)
- Time waste
- Lack of documentation
- Pair programming can take longer
- Code over design
- Lack of long term vision

Agile Methodologies Challenges

Major Problems for Agile Methodologies

1. The informality of agile development is incompatible with the legal approach to contract definition that is commonly used in companies
2. Agile methods are most appropriate for new software development rather than for software maintenance. Yet the majority of software costs in large companies come from maintaining their existing software systems.

There is no clear consensus on the suitability of agile methods for software maintenance

Choosing a Process Model

Remember:

- There is no best software process or set of software processes.
- No software process works well for every project.
- Software processes must be selected, adapted, re-adjusted and applied as appropriate for each project, for each team and for each organizational context.